

```
!pip install -U langchain langchain-groq tavily-python requests rich
```

Show hidden output

```
from langchain_groq import ChatGroq
from langchain_core.messages import HumanMessage, ToolMessage
from langchain.tools import tool
from rich import print
from tavily import TavilyClient
import os
from google.colab import userdata

import requests
```

```
@tool
def get_weather(city:str)->str:
    """Get current wheather of a {city}"""
    API_KEY=userdata.get("OPENWEATHER_API_KEY")
    url=url = f"http://api.openweathermap.org/data/2.5/weather?q={city},IN&appid={API_KEY}&units=metric"
    response=requests.get(url)
    data=response.json()
    if str(data.get("cod")) != "200":
        return f"error data couldn't found"
    temp=data["main"]["temp"]
    desc=data["weather"][0]["description"]

    return f"weather in {city} : {desc} {temp}"
```

```
tavily_client=TavilyClient(
api_key=userdata.get("TAVILY_API_KEY"))
```

```
@tool
def get_news(city:str)->str:
    """Get the latest news about {city}"""
    response=tavily_client.search(
        query=f"latest news in {city}",
        search_depth="basic",
        max_reult=3

    )
    results=response.get("results", [])
    if not results:
        return f"No news found {city}"
    news_list=[]
    for r in results:
        title=r.get("title", "no title")
        url=r.get("url", " ")
        snippet=r.get("content", " ")

        news_list.append(f"- {title}\n - {url}\n {snippet[:300]}...")
    return f" current new in {city} \n\n".join(news_list)

print(get_news.invoke("hyderabad"))
```

Show hidden output

```
from google.colab import userdata
```

```
llm=ChatGroq(model="openai/gpt-oss-120b",api_key=userdata.get('GROQ_API_KEY'))
```

```
from langchain.agents.middleware import wrap_tool_call
```

```
@tool
def web_search(query: str) -> str:
    """
    Search the internet for current information.
    Use this when the user asks for real-time or general web information.
    """

    response = tavily_client.search(
        query=query,
        search_depth="advanced",
        max_results=5
    )
```

```

results = []

for result in response["results"]:
    results.append(
        f>Title: {result['title']}\n"
        f">Content: {result['content']}\n"
        f">URL: {result['url']}"
    )

return "\n\n".join(results)

```

```

from langchain.agents import create_agent
@wrap_tool_call
def human_approval(request, handler):
    """Ask for human approval before every tool call."""
    tool_name = request.tool_call["name"]
    confirm = input(f"Agent wants to call '{tool_name}'. Approve? (yes/no): ")

    if confirm.lower() != "yes":
        return ToolMessage(
            content="Tool call denied by user.",
            tool_call_id=request.tool_call["id"]
        )

    return handler(request)

agent = create_agent(
    llm,
    tools = [get_weather, get_news, web_search],
    system_prompt= "you are a helpful city assistant.",
    middleware= [human_approval]
)

print("City Agent | type exit to quit")

while True:
    user_input = input("You : ")
    if user_input.lower() == "exit":
        break
    result = agent.invoke({
        "messages": [{"role": "user", "content": user_input}]
    })

    print("bot : ", result['messages'][-1].content )

```

[Show hidden output](#)